SKILL SWAP

Complete Technical Architecture, Line-by-Line Code Defense & Evaluator Guide

Author / Presenter: Tejas Kushwaha | **Repository:** github.com/Tejas1276/SkillSwap | **Live Site:** rad-wisp-228290.netlify.app
Target Audience: University Evaluators, Project Defense Committee, Viva Examiners, Senior Engineers

---

1. Executive Summary & 3-Minute Presentation Speech

If you are asked to introduce your project in 2 to 3 minutes, speak this exact script with confidence:

> *"Respected evaluators, today I present **Skill Swap**—a decentralized, peer-to-peer knowledge barter platform built specifically for university campus communities. In universities today, students face a fundamental dilemma: professional tutoring services and online courses are expensive and lack interactive feedback, yet within the very same campus, students possess complementary knowledge. For example, Student A may know React and need Python for machine learning, while Student B knows Python and needs React for a web project.*
>
> *Skill Swap eliminates monetary barriers by creating an **escrow-secured credit barter economy**: when you teach a 1-hour session, you earn credits; you then spend those credits to learn from peers. Our platform solves three major technical challenges: **First**, an asynchronous AI microservice calculates semantic cosine similarity over TF-IDF vector embeddings to match learners with the most credible tutor. **Second**, a dual-tier credit escrow system prevents transaction disputes. **Third**, our browser-native WebRTC audio/video classroom operates peer-to-peer with zero video server bandwidth costs, orchestrated via real-time WebSocket signaling.*
>
> *Our stack includes React 19 and Vite on the client, Node.js and Express for our REST API and WebSocket signaling, MongoDB with Mongoose and an automatic in-memory fallback for data resilience, and Python FastAPI with Scikit-learn for our AI engine."*

2. Full Technology Stack Inventory & Selection Rationale

Examiners frequently ask: *"Why did you choose Technology X instead of Technology Y?"* The following matrix outlines every technology used in Skill Swap and its architectural justification:

Technology	Role in Skill Swap	Why Chosen Over Alternatives
React 19	Frontend Single Page Application (SPA), stateful reactive UI, 9 platform screens.	Virtual DOM diffing updates only modified elements (credit counter, session timer) without page reloads. Modular component reusability beats Vanilla JS; easier state management than Angular.
Vite	Modern frontend build tool, asset bundler, and development server on port 3000.	Leverages native browser ES Modules (ESM) and esbuild written in Go. Cold server starts take <300ms compared to Webpack/Create-React-App which take 15–30 seconds.
Node.js & Express	Primary RESTful API server (port 5050), business logic, transaction controller.	Single-threaded non-blocking asynchronous event loop handles high-concurrency I/O (peer signaling and chat) without thread-pool starvation seen in multi-threaded Java or Python WSGI.
MongoDB & Mongoose	Document NoSQL database storing users, skills, sessions, and gamification logs.	JSON-like BSON documents naturally represent hierarchical data (nested `skillsToTeach` and `skillsToLearn` arrays) without expensive SQL JOINs across 5 tables.

MongoMemoryServer	In-memory MongoDB instance fallback used when local mongod daemon is offline.	Guarantees 100% presentation uptime. If evaluated on a lab PC without an active MongoDB service, the backend detects connection failure and spins up a memory replica instantly.
WebRTC (Native)	Peer-to-peer high-definition video, audio, and screen-sharing classroom.	Direct browser-to-browser UDP streaming. Media never touches the backend server, eliminating server bandwidth costs and providing sub-100ms real-time latency. Free vs Agora/Twilio.
Socket.IO	Bi-directional real-time WebSocket signaling channel for WebRTC connection setup.	Provides room multiplexing, auto-reconnection, and automatic HTTP long-polling fallback if restrictive university firewall NATs block raw WebSocket handshakes.
Python & FastAPI	Dedicated AI microservice running on port 8000 for tutor recommendations.	Python is the industry standard for ML. Running math/matrix algorithms in Python prevents CPU-intensive computations from blocking the Node.js I/O event loop.
Scikit-learn (TF-IDF)	Vector space mathematical modeling of student goals and tutor skill descriptions.	Lightweight, deterministic, and instant (<5ms inference). Avoids costly third-party API dependencies (like OpenAI API tokens) and runs fully offline during university evaluation.
JWT & bcryptjs	Stateless token-based authentication and adaptive salted password hashing.	JWT enables stateless horizontal scaling across microservices without shared session memory. bcrypt with 10 salt rounds resists rainbow table and GPU dictionary attacks.
Netlify Edge	Production public cloud hosting with global CDN distribution and SPA rewrites.	Instant deployment, global edge caching, automated HTTPS SSL certificates, and zero-maintenance static infrastructure.

3. Core Architectural Patterns & Algorithmic Models

3.1 The Escrow Credit State Machine

A critical innovation in Skill Swap is the **Double-Action Escrow Mechanism**. In a peer-to-peer barter system, trust is scarce. If credits were transferred before the session, a tutor could fail to show up. If credits were transferred after the session, a learner could disconnect without paying.

The 4-Step Escrow Protocol:

1. **Reservation State (`requested`):** When Learner books a 1-hour session costing 15 credits, the backend verifies `learner.credits >= 15`. If verified, 15 credits are deducted from the learner and held in an escrow lock inside the `Session` record. The tutor has not yet received the credits.
2. **Rejection / Cancellation (`rejected` / `cancelled`):** If the tutor rejects or the session is cancelled prior to execution, the escrowed 15 credits are automatically refunded to the learner.
3. **Completion & Release (`completed`):** When the WebRTC session finishes and both peers submit feedback, the controller releases the 15 credits from escrow and increments `tutor.credits += 15`.
4. **Conservation Invariant:** Total system credits are conserved across all transaction states: Σ **Credits**$_{\textbf{Learner}}$ **+ Credits**$_{\textbf{Escrow}}$ **+ Credits**$_{\textbf{Tutor}}$ **= Constant**.

3.2 The AI Recommendation Engine Mathematical Formula

Our Python FastAPI service (`ai-engine/main.py`) evaluates every candidate tutor using a three-factor scoring equation:

Composite Tutor Score:

$$R(tutor) = 0.50 \times S(goal, tutor) + 0.30 \times (C / 100) + 0.20 \times (Rating / 5.0)$$

Where:
• **S(goal, tutor)** = Cosine similarity of TF-IDF vectors: dot(A, B) / (||A|| × ||B||) representing semantic topic alignment.
• **C** = Dynamic Credibility Score (0 to 100), based on verified badges, session completion rate, and peer reviews.
• **Rating** = Weighted average star rating (0.0 to 5.0) awarded by previous student learners.

3.3 The WebRTC Peer-to-Peer Signaling Handshake

WebRTC allows two browsers to transmit high-definition video directly without an intermediary media server. However, before browsers can connect, they must discover each other's public IP address, port, and media capabilities. This coordination is performed by our Node.js Socket.IO server (`webrtcSignaling.js`) through a 4-step handshake:

1. **Room Discovery:** Peer A joins room `room-101`. Socket.IO tracks connected peers.
2. **SDP Offer:** Peer A creates an SDP (Session Description Protocol) offer containing video codecs, resolutions, and timing, sending it via Socket.IO to Peer B.
3. **SDP Answer:** Peer B receives the offer, sets it as remote description, generates an SDP answer, and returns it to Peer A.
4. **ICE Candidate Exchange:** Both peers query Google STUN servers (`stun:stun.l.google.com:19302`) to discover their public NAT traversal candidates. Once ICE candidates are exchanged via signaling, media streams flow directly peer-to-peer via encrypted SRTP.

4. Systematic Code Walkthrough (Line-by-Line Defense)

Here we walk through the exact source code files of Skill Swap, explaining the purpose and mechanics of each line:

■ File: backend/src/server.js (Server Bootstrapper)

```
const http = require('http');
const dotenv = require('dotenv');
dotenv.config();
const app = require('./app');
const { connectDB, disconnectDB } = require('./config/db');
const { ensureDemoUsers } = require('./config/ensureDemoUsers');
const { initWebRTCSignaling } = require('./signaling/webrtcSignaling');
const PORT = process.env.PORT || 5050;
const startServer = async () => {
  await connectDB();
  await ensureDemoUsers();
  server = http.createServer(app);
  io = initWebRTCSignaling(server);
  server.listen(PORT, () => console.log(`Running on ${PORT}`));
};
```

Explanation & Line Mechanics:
• **dotenv.config()**: Loads environment variables (.env) into Node process memory before imports run.
• **connectDB()**: Asynchronously opens the MongoDB connection with automatic fallback to memory server.
• **ensureDemoUsers()**: Idempotently verifies that test personas (Tejas, Rohan, Priya) exist with valid credentials so evaluators can test instantly without manual account setup.
• **http.createServer(app)**: Wraps Express inside a native Node HTTP server so WebSocket signaling and REST routes share port 5050.
• **initWebRTCSignaling(server)**: Attaches Socket.IO to the HTTP server for real-time video signaling.
• **process.on('SIGINT', shutdown)**: Intercepts Ctrl+C signals to gracefully close WebSocket connections and database pools without data corruption.

■ File: backend/src/config/db.js (Resilient Database Connection)

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI, { serverSelectionTimeoutMS: 2000 });
    console.log(`Connected to MongoDB: ${conn.connection.host}`);
  } catch (err) {
    const { MongoMemoryServer } = require('mongodb-memory-server');
    mongoServer = await MongoMemoryServer.create();
    await mongoose.connect(mongoServer.getUri());
    console.log('Connected to fallback in-memory MongoDB');
  }
};
```

Explanation & Line Mechanics:
• **serverSelectionTimeoutMS: 2000**: Fails fast after 2 seconds if a local MongoDB service is not running on 127.0.0.1:27017, preventing long hangs during startup.
• **catch (err)**: Catches connection errors (e.g., ECONNREFUSED) gracefully.
• **MongoMemoryServer.create()**: Dynamically starts an in-memory MongoDB replica binary. This ensures that even if you evaluate the project on an external examiner laptop with no database installed, the server boots flawlessly!

■ File: backend/src/models/User.js (Password Cryptography & Security)

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
userSchema.methods.comparePassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};
```

Explanation & Line Mechanics:
• **userSchema.pre('save')**: Mongoose pre-save middleware lifecycle hook that intercepts document insertion/update before it reaches disk.
• **isModified('password')**: Prevents rehashing an already-hashed password when only updating bio, credits, or skills.
• **bcrypt.genSalt(10)**: Generates a cryptographically strong 10-round salt to defend against rainbow table attacks.
• **bcrypt.hash()**: Performs one-way cryptographic hashing.
• **comparePassword()**: Uses constant-time string comparison to prevent timing side-channel attacks during authentication.

■ File: backend/src/controllers/authController.js (Registration & JWT Issuance)

```
const register = async (req, res, next) => {
  const { name, email, password, bio } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) return res.status(409).json({ message: 'User already exists' });
  const user = await User.create({ name, email, password, credits: 100, credibilityScore: 70.0 });
  await Gamification.create({ user: user._id, type: 'bonus_received', amount: 100 });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.status(201).json({ success: true, token, user });
};
```

Explanation & Line Mechanics:

• **User.findOne({ email })**: Enforces unique user identity.
• **credits: 100**: Seeds new students with 100 welcome credits so they can immediately book learning sessions without waiting.
• **Gamification.create()**: Creates an audit log of the welcome bonus in the ledger.
• **jwt.sign(...)**: Creates a cryptographically signed JSON Web Token containing the user ID payload and 7-day expiration.
• **res.status(201)**: Returns HTTP 201 Created status code conforming to RESTful standards.

■ File: backend/src/signaling/webrtcSignaling.js (WebRTC Signaling Controller)

```
socket.on('join-room', ({ roomId, userName, userId }) => {
  const room = rooms.get(roomId) || new Map();
  if (room.size >= 2) return socket.emit('room-full');
  socket.join(roomId);
  room.set(socket.id, { userName, userId });
  rooms.set(roomId, room);
  if (room.size === 2) {
    const peers = Array.from(room.entries());
    socket.to(peers[0][0]).emit('peer-joined', { peerSocketId: socket.id, userName });
  }
});
socket.on('offer', ({ targetSocketId, offer }) => io.to(targetSocketId).emit('offer', { offer,
senderSocketId: socket.id }));
socket.on('ice-candidate', ({ targetSocketId, candidate }) => io.to(targetSocketId).emit('ice-candidate', {
candidate }));
```

Explanation & Line Mechanics:

• **room.size >= 2**: Enforces 1-on-1 private tutoring session constraint; third parties cannot eavesdrop.
• **socket.join(roomId)**: Groups sockets into a dedicated Socket.IO room channel.
• **peer-joined**: Fires when the second peer enters, signaling Peer 1 to generate the WebRTC SDP offer.
• **io.to(targetSocketId).emit('offer' / 'ice-candidate')**: Relays connection parameters directly between the two browser tabs without saving or processing media on the server.

■ File: ai-engine/main.py (FastAPI AI Matchmaking Microservice)

```
@app.post('/api/ai/match-tutors')
async def match_tutors(req: MatchRequest):
  tutor_texts = [f"{t.name} {' '.join(s.skill for s in t.skillsToTeach)}" for t in req.tutors]
  corpus = [req.goalSkill] + tutor_texts
  vectorizer = TfidfVectorizer(stop_words='english')
  tfidf_matrix = vectorizer.fit_transform(corpus)
  similarities = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:]).flatten()
  for idx, tutor in enumerate(req.tutors):
    sim = similarities[idx]
    score = 0.5 * sim + 0.3 * (tutor.credibilityScore / 100) + 0.2 * (tutor.rating.average / 5.0)
    ranks.append((score, tutor))
  ranks.sort(reverse=True, key=lambda x: x[0])
  return {'recommendations': ranks};
```

Explanation & Line Mechanics:

• **TfidfVectorizer**: Converts textual skill descriptions into sparse numeric vectors based on Term Frequency × Inverse Document Frequency, penalizing common filler words and highlighting unique skills.

• **cosine_similarity**: Computes the geometric cosine angle between the student's learning goal vector and each tutor's skill vector.

• **Weighted Ranking (0.5 / 0.3 / 0.2)**: Balances semantic relevance (50%) with tutor credibility (30%) and historical student ratings (20%).

■ File: frontend/src/context/AuthContext.jsx (Resilient Auth State)

```
const login = async (email, password) => {
  try {
    const data = await api.auth.login({ email, password });
    localStorage.setItem('skillswap_token', data.token);
    setUser(data.user);
    return data.user;
  } catch (err) {
    if (DEMO_FALLBACK_USERS[email.toLowerCase()] || err.message.includes('fetch')) {
      const fallback = DEMO_FALLBACK_USERS[email.toLowerCase()];
      localStorage.setItem('skillswap_token', 'mock-demo-token');
      setUser(fallback);
      return fallback;
    }
    throw err;
  }
};
```

Explanation & Line Mechanics:

• **localStorage.setItem**: Persists the JWT token so user stays logged in across page reloads and browser restarts.

• **catch (err) Static Fallback**: If the application is hosted on public static platforms like Netlify before the backend microservice is linked, the context seamlessly serves pre-configured persona profiles (Tejas, Rohan, Priya). This guarantees zero blank screens or connection crashes during live presentations!

■ File: frontend/src/pages/VideoRoomPage.jsx (WebRTC Audio/Video Engine)

```
const startCall = async () => {
  const stream = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });
  localVideoRef.current.srcObject = stream;
  peerConnection.current = new RTCPeerConnection({ iceServers: [{ urls: 'stun:stun.l.google.com:19302' }] });
  stream.getTracks().forEach(track => peerConnection.current.addTrack(track, stream));
  peerConnection.current.ontrack = (event) => { remoteVideoRef.current.srcObject = event.streams[0]; };
  peerConnection.current.onicecandidate = (event) => {
    if (event.candidate) socket.emit('ice-candidate', { candidate: event.candidate, targetSocketId });
  };
};
```

Explanation & Line Mechanics:

• **navigator.mediaDevices.getUserMedia**: Requests browser camera and microphone hardware permissions.

• **RTCPeerConnection**: Creates the WebRTC peer connection object with Google's public STUN server for NAT traversal.

• **addTrack()**: Injects the student's camera video and microphone audio tracks into the peer connection.

• **ontrack**: Event listener that triggers when the remote peer's media stream arrives, attaching it directly to the video element for instant playback.

5. Professor & Examiner Deep-Drill Defense (Viva Q&A)

Examiners will challenge you on architecture, edge cases, scalability, and security. Here are the top 15 questions and the high-scoring answers you should deliver:

Q1: What happens if a student runs out of credits? How do they get more without paying real money?

Skill Swap is a pure barter economy. If a student has 0 credits, they cannot pay real currency. Instead, they earn credits by offering their own skills as a tutor. When they conduct a 1-hour session teaching another peer, they receive the session's hourly credits into their balance. New students also receive a 100-credit welcome bonus upon registration.

Q2: How does your system prevent two peers from cheating and inflating each other's credits?

We enforce three safeguards: First, credit transactions are zero-sum; credits transferred to the tutor are deducted from the learner. No new credits are minted during a peer session. Second, our anti-collusion algorithm in `feedbackController.js` caps repeat reviews between the same two user IDs. Third, the AI Credibility Score incorporates total unique peers taught, penalizing circular reciprocal farming.

Q3: Why use WebRTC instead of embedding Zoom SDK, Agora, or Twilio?

Commercial SDKs like Agora or Twilio charge per participant-minute (e.g., $0.004/min), which becomes unaffordable for a free student platform at scale. WebRTC is an open W3C/IETF standard native to all modern browsers (Chrome, Edge, Safari, Firefox). Because media flows peer-to-peer over UDP, our university server incurs zero media streaming bandwidth costs.

Q4: What if a student is behind a strict college firewall that blocks peer-to-peer UDP connections?

Our WebRTC implementation queries STUN servers (`stun.l.google.com:19302`) to resolve public NAT bindings. For symmetrical enterprise NATs that block direct P2P connections, the WebRTC standard defines a TURN (Traversal Using Relays around NAT) relay fallback server (such as Coturn). Furthermore, our Socket.IO signaling layer automatically falls back to HTTP long-polling if WebSockets are blocked.

Q5: Why did you separate the AI engine into Python instead of running ML inside Node.js?

Node.js runs on a single-threaded event loop. If we execute heavy numerical matrix computations (such as TF-IDF vectorization and cosine similarity across thousands of tutors) directly in Node.js, the event loop would block, freezing all incoming HTTP requests and WebSocket signaling. By decoupling ML into a Python FastAPI microservice, we leverage Python's optimized C-extensions (NumPy/BLAS) while keeping Node.js responsive.

Q6: Why did you choose MongoDB instead of MySQL or PostgreSQL?

In Skill Swap, a user document contains flexible hierarchical structures: `skillsToTeach` (each with proficiency, rate, verification status), `skillsToLearn` (with target level and notes), and `badges`. In a relational database, querying a user profile with all their skills, badges, and reviews requires multi-table JOIN operations that degrade read performance. In MongoDB, a single BSON document fetch returns the entire profile in sub-millisecond time.

Q7: How do you protect against SQL / NoSQL Injection attacks?

We use Mongoose schemas with strict data typing and query parameterization. User input is never concatenated into raw database strings. Furthermore, in `authController.js`, queries sanitize inputs, and Mongoose strips out non-schema operators like `$where` or `$gt` from request payloads.

Q8: What is the purpose of bcrypt salt rounds?

A rainbow table is a precomputed lookup table of reversed password hashes. A cryptographic salt is a random 16-byte sequence appended to the plaintext password before hashing. Even if two users share the password 'password123', their bcrypt hashes will be completely different. The 10 salt rounds exponentially increase the computational cost, rendering brute-force attacks infeasible.

Q9: How do you manage user sessions across multiple servers without storing session cookies?

We use stateless JSON Web Tokens (JWT). When a user logs in, the backend signs a payload containing `{ id: user._id }` using our secret key. The client includes this token in the `Authorization: Bearer ` header on every subsequent request. The server verifies the cryptographic signature mathematically without needing to query a shared session store (like Redis), allowing seamless horizontal scaling.

Q10: What happens if a tutor accepts a session booking and never joins the call?

The learner's credits are safely locked in the `Session` escrow record. If the tutor fails to join or the meeting room expires without a peer connection handshake, the session status updates to `cancelled` and our controller automatically releases the escrow back to the learner's credit balance.

Q11: Why is Vite faster than Webpack or Create-React-App?

Webpack compiles and bundles the entire application codebase before starting the development server. As the app grows, boot times take 30–60 seconds. Vite does not bundle in development: it starts the server immediately and lets the browser request modules natively via ES Modules (`import/export`) on demand. Dependencies are pre-bundled using `esbuild`, which is written in Go and runs 10 to 100 times faster than JavaScript-based bundlers.

Q12: How does the system handle concurrent bookings for the same tutor time slot?

In `learnerController.js`, session booking uses MongoDB atomic queries. Before creating a booking, the query checks whether an existing session exists for the tutor with `status: { $in: ['accepted', 'requested'] }` matching the requested time window. If a conflict exists, the server rejects the request with HTTP 409 Conflict.

Q13: What is the difference between WebSockets and WebRTC?

WebSockets is a client-server TCP protocol used for bi-directional text and JSON messaging (such as our chat and signaling messages). WebRTC is a peer-to-peer UDP protocol used for real-time streaming of high-throughput audio and video directly between client browsers without passing through the server.

Q14: How does your frontend work if the backend server is temporarily down?

In `frontend/src/context/AuthContext.jsx`, our API client includes an offline resilience fallback. If a network fetch fails, the client gracefully falls back to pre-configured demonstration personas (Tejas, Rohan, Priya) stored in memory, allowing all 9 frontend screens, dashboard KPIs, and navigation tabs to remain fully navigable during demonstrations.

Q15: What are the future enhancements planned for Skill Swap?

1. Integration of decentralized smart contracts for cross-university credit verification.
2. Automated AI meeting summarization using local Whisper speech-to-text models.
3. Interactive collaborative whiteboard canvas shared in real-time over WebRTC data channels.

End of Defense Manual | *Skill Swap Architecture & Engineering Defense Manual © 2026*